

HackJudge Hackathon Suite: Technical Project Report

This report provides a comprehensive, human-readable overview of the four core projects comprising the **HackJudge Hackathon Suite**. This ecosystem is designed to run live hackathons, manage player registrations, execute real-time ice-breaking quizzes, automatically grade submitted GitHub repositories using AI-driven criteria, and display real-time standings on a visual leaderboard.

Suite Architecture Overview

The suite operates as a unified platform where individual components communicate via REST APIs and real-time database syncing (Supabase/PostgreSQL):

```
<div class="flow-box">
  <div class="flow-title">Participant Submission Flow</div>
  <p style="margin-bottom: 10px; margin-left: 10px;">
    <strong>Participants</strong> submit their repository URL to the <strong>Participant Portal</strong>.
    &rarr; Portal validates the deadline and forwards it to the <strong>Judging Engine</strong>.
    &rarr; Engine runs async Git history analytics and LLM criteria evaluations
    &rarr; Saves results to the <strong>Neon / Supabase PostgreSQL database</strong>.
  </p>
  <div class="flow-title">Real-time Leaderboard Flow</div>
  <p style="margin-bottom: 10px; margin-left: 10px;">
    <strong>Supabase Postgres Changes Channel</strong> triggers instant broadcasts on score updates.
    &rarr; <strong>Leaderboard UI</strong> catches changes and updates rankings and countdown timers.
  </p>
  <div class="flow-title">Ice Breaking Trivia Flow</div>
  <p style="margin-left: 10px; margin-bottom: 0;">
    Coordinator manages the trivia coordinates in the <strong>Ice Breaking Control App</strong>.
    &rarr; Players submit answers in real time
    &rarr; Answer transactions are processed atomically and broadcast to players via <strong>Sequencer</strong>.
  </p>
</div>
```

1. HackJudge Participant Portal

Purpose

The **HackJudge Participant Portal** acts as the direct participant-facing interface. It allows hackathon teams to submit their GitHub repository URLs and deployment links before the deadline. It coordinates directly with the **Judging Engine** backend for evaluation and updates the participant records in **Supabase** once the analysis is completed.

Tech Stack

- **Framework:** Next.js 16.2.6 (App Router)
- **Language:** TypeScript (v5)
- **Styling:** TailwindCSS v4 with `@tailwindcss/postcss`
- **Icons:** Lucide React
- **Client Database Integration:** `@supabase/supabase-js`
- **Package Manager:** npm

Database Architecture

- **Database Provider:** Supabase (PostgreSQL)

Key Tables:

- `submissions`: Stores participant details, repository URLs, team names, deployment links, and final scores.
- `config`: Stores configuration keys, such as the countdown deadline (`end_time`) and whether the leaderboard is frozen (`is_frozen`).

API Routes & Functionality

GET `/api/config`

- **Description:** Fetches global hackathon settings (freeze state and end-time deadline) from Supabase.

POST `/api/config`

- **Description:** Allows administrators to update deadline timestamps or freeze/unfreeze the leaderboard. Requires authentication.

POST `/api/submit`

- **Description:** Receives a project submission (`github_url`, `team_name`, `deployment_link`). Validates that the deadline has not passed, proxies the analysis request to the Judging Engine (`/api/analyze`), and registers the state. If the repository was already graded, it immediately returns the cached scores.

GET `/api/submit`

- **Description:** A polling endpoint invoked by the client-side UI to fetch background analysis status. Once complete, it saves the scores directly to Supabase and returns the payload.

POST `/api/admin/verify`

- **Description:** Validates the entered admin password.

POST `/api/admin/action`

- **Description:** Enables admins to delete specific submissions (sending a delete request to both Supabase and the Judging Engine cache) or clear all records.

Configuration & Environment Keys

- `VITE_JUDGING_ENGINE_URL`: The base address of the Judging Engine server.
 - `VITE_SUPABASE_URL`: The Supabase project endpoint.
 - `VITE_SUPABASE_ANON_KEY`: The public API token used to authenticate Supabase requests.
 - `VITE_ADMIN_PASSWORD`: Secret password used for admin panel access.
 - `VITE_HACKATHON_END_TIME`: The default datetime timestamp marking the end of the hackathon.
-

2. Ice Breaking

Purpose

The **Ice Breaking** application is a real-time, highly concurrent trivia game used to energize participants during the hackathon. It supports instant player registration, real-time question releases, rapid answer submissions, and instant standings updates broadcast via Server-Sent Events (SSE).

Tech Stack

- **Framework**: Next.js 16.2.6 (App Router)
- **Language**: TypeScript (v5)
- **Styling**: TailwindCSS v4
- **Database Driver**: Direct PostgreSQL Client (pg pool connection)
- **Event System**: Node.js Event Emitter (for Server-Sent Events broadcasting)
- **Optimization**: Custom in-memory Cache layer with Time-To-Live (TTL) and Request Coalescing (deduplication) to handle high-concurrency requests safely.

Database Architecture

- **Database Provider**: PostgreSQL (hosted on Neon Serverless Postgres)

Key Tables:

- `players`: Tracks player names, unique alphanumeric University Student Numbers (USN), cumulative scores, correct answers count, and completion status.
- `player_answers`: Logs every individual answer submission, storing the selected options, points earned, correctness, and exact timestamps.
- `quiz_session_state`: Coordinates the live quiz, tracking active question indices, timer start times, durations, lock status, and whether the quiz has ended.
- **Database Indexes**: Configured on `usn`, `(player_usn, question_index)`, `question_index`, and `(score, correct_count)` to keep reads lightning-fast under heavy submission volume.

API Routes & Functionality

POST /api/register

- **Description:** Registers or updates players in the database using their name and alphanumeric USN (e.g., 1BY25CS094).

POST /api/submit-answer

- **Description:** Records players' answers to MCQ (10 points) or emoji questions (5 points). Enforces validation rules (verifying player registry, active live question index, timer limits, and session locks) and updates points atomically using a single CTE database transaction.

GET /api/session-state

- **Description:** Returns the active question state, lock configuration, remaining time, and whether the player's USN is registered. Uses request coalescing to protect the database.

GET /api/leaderboard

- **Description:** Retrieves player standings sorted by score (primary) and correct answer count (secondary) with brief in-memory cache layers.

GET /api/sse

- **Description:** Establishes a persistent SSE stream to broadcast live session updates and leaderboard refreshes directly to connected clients.

POST /api/admin/control

- **Description:** Allows coordinators to advance questions, start/stop timers, lock submissions, reveal answers, drop players, or hide the leaderboard.

POST /api/admin/reset

- **Description:** Resets the entire database, clearing players and restoring the default quiz state.

Configuration & Environment Keys

- `DATABASE_URL`: Connection URL for Neon Serverless PostgreSQL.
- `ADMIN_PASSWORD`: Secret token required to coordinate and manage the live game.

3. Judging Engine

Purpose

The **Judging Engine** is the core automation layer that grades GitHub repositories. It fetches repository details, commit activity, issues, and key source codes, runs cheat-detection tests to flag pre-written projects, and evaluates the project against 7 distinct parameters using AI models.

Tech Stack

- **Framework:** Next.js 14.2.35 (App Router)
- **Language:** TypeScript
- **ORM:** Prisma Client (@prisma/client)
- **Integrations:** Octokit REST Client (GitHub API interactions), Google Gemini API, Groq API, and Anthropic API.
- **Deployment System:** Koyeb (.koyeb.yaml configuration)

Database Architecture

- **Database Provider:** PostgreSQL (Prisma ORM integration)

Key Model:

- **Submission:** Captures repository metadata (stars, forks, languages, commits, lines of code), total score, 7-parameter score cards, textual AI reviews, cheat flagging reasons, earliest dependency dates, and background analysis status.

API Routes & Functionality

POST /api/analyze

- **Description:** Submits a repository (or an array of URLs for batch grading) for analysis. Runs controlled workers (up to 4 concurrently, 500ms pacing delay) to process repositories in the background, ensuring GitHub rate limits are not exceeded.

GET /api/result/[id]

- **Description:** Retrieves the completed evaluation, scoring grades, and review text for a specific submission ID.

GET /api/results

- **Description:** Retrieves all submissions.

DELETE /api/results

- **Description:** Deletes a specific repository evaluation or clears the database.

POST /api/results/push

- **Description:** Pushes completed scores to the primary leaderboard.

POST /api/reflag

- **Description:** Reruns cheat-detection checks on existing database entries.

Scoring Weights

The engine automatically detects if a project is standard or headless (backend, database, library, CLI) and adjusts parameter weights accordingly:

| Parameter | Standard Weight | Headless Weight | Description | |---|---|---|---| | **Security** | 10% | 10% | Secrets handling, inputs validation, config segregation. | | **Code Quality** | 15% | 25% | Modular design, error prevention, typing safety. | | **Innovation** | 20% | 20% | Idea complexity, creative mechanics, integrations. | | **UI / UX** | 30% | 10% | Styling, polish, accessibility / CLI output layout. | | **Backend** | 10% | 25% | API stability, DB modeling, performance design. | | **Commits** | 10% | 10% | Branching structure, commit descriptions. | | **Documentation** | 5% | 5% | README clarity, setup guidelines, inline guides. |

Automated Anti-Cheat Detection

To ensure fairness, the engine implements 7 checks:

1. **Creation Date Check:** Checks if the repository was created before the start date.
2. **Prior Commits Check:** Scans commit history for dates preceding the event start date.
3. **Rebase/Force-Push Laundering Check:** Detects if code was written pre-event but rebased/cherry-picked during the hackathon.
4. **Dependency Lockfile Date:** Inspects the creation date inside `package-lock.json` or `yarn.lock`.
5. **Bulk Upload Burst:** Flags repositories where 5+ commits are uploaded in a tight 20-minute window.
6. **Hardcoded Date Detection:** Scans file contents for dates/timestamps preceding the event.
7. **Single Commit Size:** Flags repositories with only 1 commit that contains huge code additions.

Configuration & Environment Keys

- `GEMINI_API_KEYS`: A pool of comma-separated Gemini API keys used to distribute AI request load.
- `GROQ_API_KEYS`: A pool of comma-separated Groq API keys used to evaluate code via Llama-3.3-70b.
- `GITHUB_TOKENS`: A pool of GitHub PAT tokens used to bypass GitHub REST API rate limits.
- `DATABASE_URL`: Connection string connecting Prisma to the PostgreSQL database.
- `EVENT_DATE`: The start datetime timestamp used for all anti-cheat date checks.
- `TZ`: The system timezone (e.g., Asia/Kolkata).
- `ANTHROPIC_API_KEY`: Fallback API key for Claude Sonnet.
- `USE_OLLAMA` / `OLLAMA_MODEL` / `OLLAMA_HOST`: Config variables for running evaluations locally/offline.
- `CUSTOM_LLM_KEY` / `CUSTOM_LLM_URL` / `CUSTOM_LLM_MODEL`: Settings for running queries through custom open proxies.

4. Leaderboard

Purpose

The **Leaderboard** is a visual scoreboard displaying participant rankings in real time. It features a countdown timer, podium displays for the top 3 teams, rank cards, and live updates showing new entries and updated scores as they are saved.

Tech Stack

- **Framework:** React 19.2.6 (Vite App)
- **Language:** TypeScript
- **Styling:** Modern CSS variables with custom keyframes and staggered animations
- **Database integration:** `@supabase/supabase-js` (real-time channel listener)

Database Architecture

- **Database Provider:** Supabase (shared table access)

Tables Synced:

- `submissions`: Read-only queries to fetch scores ordered by rank descending.
- `config`: Periodic polling to update the hackathon deadline countdown.

Core Functionality

- **Real-time Synchronization:** Uses Supabase Postgres Changes Channel (`realtime_submissions`) to trigger instant score updates. Includes a 10-second polling fallback.
- **Countdown Clock:** Displays the time remaining in the hackathon, updated via periodic syncs with the admin configuration.
- **Podium & List Render:** Displays the top 3 teams on a visual podium, followed by a ranked table.
- **Interactive Tagging:** Automatically categorizes repositories with badges (e.g., AI, WEB, API, DEV) by scanning keywords in repository names.
- **User Highlighting:** Employs client-side checks to highlight the current user's team row ("YOU" badge) to personalize the display.

Configuration & Environment Keys

- `VITE_SUPABASE_URL`: The Supabase project endpoint.
- `VITE_SUPABASE_ANON_KEY`: The public API token used to query Supabase.
- `VITE_HACKATHON_END_TIME`: Fallback countdown timestamp.
- `VITE_CURRENT_USER_REPO`: The GitHub URL of the team viewing the dashboard, used to highlight their rank.